

STA 445 Assignment 1

Kylee Peshlakai

2026-02-17

```
library(tidyverse)
```

Problem 1

Create a vector of three elements (2,4,6) and name that vector `vec_a`. Create a second vector, `vec_b`, that contains (8,10,12). Add these two vectors together and name the result `vec_c`. Print only the result `vec_c`. Do not use the print command.

```
vec_a <- c(2,4,6)
vec_b <- c(8,10,12)
vec_c <- vec_a + vec_b
vec_c
```

```
## [1] 10 14 18
```

Problem 2

Create a vector, named `vec_d`, that contains only two elements (14,20). Add this vector to `vec_a` and show the result. What is the result and what do you think R did (look up the recycling rule using Google)? What is the warning message that R gives you? The result is (16, 24, 20). What R does is it continues to add the smaller length vector, `vec_d`, to the bigger length vector, `vec_a`, repeatedly, until everything from the `vec_a` is added. The warning message that R gives us is “longer object length is not a multiple of shorter object length”, which means that the `vec_a` is the longer object length, is not equal to the `vec_d`.

```
vec_d <- c(14,20)
results2 <- vec_d + vec_a
```

```
## Warning in vec_d + vec_a: longer object length is not a multiple of shorter
## object length
```

```
results2
```

```
## [1] 16 24 20
```

Problem 3

Next add 5 to the vector `vec_a` and show the result. What is the result and what did R do? Why doesn't R give you a warning message similar to what you saw in the previous problem? The results is (7,9,11). What R did was add 5 to each number in the vector. It doesn't give a warning message because its a number being added to the vector, not a vector adding to another vector.

```
results3 <- 5 + vec_a
results3
```

```
## [1] 7 9 11
```

Problem 4

Generate the vector of integers $\{1, 2, \dots, 5\}$ in two different ways.

- a. First using the `seq()` function

```
seq(from=1, to=5, by=1)
```

```
## [1] 1 2 3 4 5
```

- b. Using the `a:b` shortcut.

```
1:5
```

```
## [1] 1 2 3 4 5
```

Problem 5

Generate the vector of even numbers $\{2, 4, 6, \dots, 20\}$

- a. Using the `seq()` function

```
seq(from=2, to=20, by=2)
```

```
## [1] 2 4 6 8 10 12 14 16 18 20
```

- b. Using the `a:b` shortcut and some subsequent algebra.

```
1:10 *2
```

```
## [1] 2 4 6 8 10 12 14 16 18 20
```

Problem 6

Generate a vector of 21 elements that are evenly spaced between 0 and 1 using the `seq()` command and name this vector `x`. Print the vector `x`.

```
x <- seq(from=0, to=1, length.out=21)
x
```

```
## [1] 0.00 0.05 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45 0.50 0.55 0.60 0.65 0.70
## [16] 0.75 0.80 0.85 0.90 0.95 1.00
```

Problem 7

Generate the vector $\{2, 4, 8, 2, 4, 8, 2, 4, 8\}$ using the `rep()` command to replicate the vector `c(2,4,8)`.

```
rep(c(2, 4, 8), 3)
```

```
## [1] 2 4 8 2 4 8 2 4 8
```

Problem 8

Generate the vector $\{2, 2, 2, 2, 4, 4, 4, 4, 8, 8, 8, 8\}$ using the `rep()` command. You might need to check the help file for `rep()` to see all of the options that `rep()` will accept. In particular, look at the optional argument `each=`.

```
rep(c(2, 4, 8), each=4)
```

```
## [1] 2 2 2 2 4 4 4 4 8 8 8 8
```

Problem 9

In this problem, we will work with the matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 12 & 14 & 16 & 18 & 20 \\ 22 & 24 & 26 & 28 & 30 \end{bmatrix}$$

- Follow the prompts below to create the matrix in two ways and save the resulting matrix as `M`.
- Create the matrix using some combination of the `seq()` and `matrix()` commands. Print `M`.

```
M <- matrix(seq(from=2, to=30, by=2), nrow=3, ncol=5, byrow = TRUE)
M
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    2    4    6    8   10
## [2,]   12   14   16   18   20
## [3,]   22   24   26   28   30
```

- ii. Create the same matrix by some combination of multiple `seq()` commands and either the `rbind()` or `cbind()` command. Print M

```
seq1 <- seq(2,10, by=2)
seq2 <- seq(12,20, by=2)
seq3 <- seq(22,30, by=2)
M <- rbind(seq1, seq2, seq3)
M
```

```
##      [,1] [,2] [,3] [,4] [,5]
## seq1    2    4    6    8   10
## seq2   12   14   16   18   20
## seq3   22   24   26   28   30
```

- b. Extract the second row out of M.

```
M[2,]
```

```
## [1] 12 14 16 18 20
```

- c. Extract the element in the third row and second column of M.

```
M[3,2]
```

```
## seq3
##    24
```

Problem 10

The following code creates a `data.frame` and then has two different methods for removing the rows with NA values in the column `Grade`. Explain the difference between the two. According to R, the `which` function gives the TRUE indices. So, “-which” must give the FALSE indices of NA values in the data frame, which removes the NA values and keeps other values. According to R, the `is.na` function indicates which elements are missing. So, “!is.na” gives “NOT NA” value. With this, it removes the NA values and keeps the other values. Each of these methods finds the non-NA values, which then grabs these values and leave behind the NA values.

```
df <- data.frame(name= c('Alice','Bob','Charlie','Daniel'),
                  Grade = c(6,8,NA,9))

df[ -which( is.na(df$Grade) ), ]

df[ which( !is.na(df$Grade) ), ]
```

Problem 11

Create and manipulate a list.

- a. Create a list named `my.test` with elements `x = c(4,5,6,7,8,9,10)` + `y = c(34,35,41,40,45,47,51)` + `slope = 2.82` + `p.value = 0.000131`

```
x = c(4,5,6,7,8,9,10)
y = c(34,35,41,40,45,47,51)
slope = 2.82
p.value = 0.000131
my.test <- list(X=x, Y=y, Slope=slope, P.Value=p.value)
str(my.test)
```

```
## List of 4
## $ X      : num [1:7] 4 5 6 7 8 9 10
## $ Y      : num [1:7] 34 35 41 40 45 47 51
## $ Slope  : num 2.82
## $ P.Value: num 0.000131
```

To show you have created your list correctly, type use `str()` to show the structure of the list.

- b. Extract the second element in the list.

```
my.test[[2]]
```

```
## [1] 34 35 41 40 45 47 51
```

- c. Extract the element named `p.value` from the list.

```
my.test[['P.Value']]
```

```
## [1] 0.000131
```